KEE Audio CODEC

Design and Implementation

A Custom Audio Compression Format
Using FFT-Based Downsampling

KEE CODEC Project

April 17, 2026

Contents

1 Introduction to Audio Codecs **3**
- 1.1 What Is a Codec? . . . 3
- 1.2 Why Codecs Matter . . . 3
- 1.3 Lossless vs. Lossy Compression . . . 3
- 1.4 Key Concepts . . . 4
  - 1.4.1 Sampling Rate and the Nyquist Theorem . . . 4
  - 1.4.2 Quantisation and Bit Depth . . . 4
  - 1.4.3 The Fast Fourier Transform (FFT) . . . 4
- 1.5 Common Audio Codecs . . . 4
- 1.6 Where the KEE Codec Fits . . . 5

2 Core Components of an Audio Codec **6**
- 2.1 Signal Analysis (Frequency Domain) . . . 6
- 2.2 Filtering . . . 6
- 2.3 Downsampling and Upsampling . . . 6
  - 2.3.1 Downsampling (Decimation) . . . 6
  - 2.3.2 Upsampling (Interpolation) . . . 7
- 2.4 Metadata and Container Formats . . . 7
- 2.5 Integrity Verification . . . 7
- 2.6 Generic Encode / Decode Pipeline . . . 7

3 KEE CODEC Project Overview **8**
- 3.1 Project Goals and Motivation . . . 8
- 3.2 High-Level Architecture . . . 8
- 3.3 Technology Stack . . . 8
- 3.4 Project Directory Structure . . . 9

4 Detailed Component Descriptions **10**
- 4.1 Audio I/O (`src/io_audio.py`) . . . 10
- 4.2 Metadata (`src/metadata.py`) . . . 10
- 4.3 Auto Downsample Factor (`src/downsample_factor.py`) . . . 11
- 4.4 Resampling (`src/resample.py`) . . . 12
  - 4.4.1 Low-Pass Filter . . . 12
  - 4.4.2 Decimate . . . 12
  - 4.4.3 Upsample . . . 12
- 4.5 Custom Format (`src/custom_format.py`) . . . 13
- 4.6 Encoding Pipeline (`src/encode.py`) . . . 13
- 4.7 Decoding Pipeline (`src/decode.py`) . . . 14

5 Development Phases **16**
5.1 Phase 1: Core CODEC . . . 16
5.1.1 Stage 1.1 – Project Setup and Dependencies . . . 16
5.1.2 Stage 1.2 – Input Handling and Analysis . . . 16
5.1.3 Stage 1.3 – Downsampling (Encoding) . . . 16
5.1.4 Stage 1.4 – Custom Format Design . . . 17
5.1.5 Stage 1.5 – Encoding Pipeline . . . 17
5.1.6 Stage 1.6 – Upsampling (Decoding) . . . 17
5.1.7 Stage 1.7 – Decoding Pipeline . . . 17
5.2 Phase 2: CLI and Scripts . . . 17
5.2.1 Stage 2.1 – Command-Line Interface . . . 17
5.2.2 Stage 2.2 – Metafile-Only Option . . . 18
5.3 Phase 3: Web UI . . . 18
5.3.1 Stage 3.1 – Landing and Encode/Decode Entry . . . 18
5.3.2 Stage 3.2 – Encode Flow . . . 18
5.3.3 Stage 3.3 – Decode Flow . . . 18
5.3.4 Stage 3.4 – Backend . . . 19
5.4 Phase 4: Quality and Robustness . . . 19
5.4.1 Stage 4.1 – Validation and Tests . . . 19
5.4.2 Stage 4.2 – Error Handling and Limits . . . 19
5.5 Phase 5: Documentation and Enhancements . . . 19
5.5.1 Stage 5.1 – Documentation . . . 19
5.5.2 Stage 5.2 – Backlog Alignment and Future Work . . . 20

6 The `.kee` File Format Specification **21**
6.1 Binary Layout . . . 21
6.2 Metadata JSON Schema . . . 21
6.3 Payload Format . . . 21
6.4 Format Diagram . . . 22

7 Usage Guide **23**
7.1 Installation . . . 23
7.2 CLI Encoding . . . 23
7.3 CLI Decoding . . . 23
7.4 Web UI . . . 24
7.4.1 Encode Workflow . . . 24
7.4.2 Decode Workflow . . . 24
7.5 Running Tests . . . 24

8 Conclusion and Future Work **25**
8.1 Summary . . . 25
8.2 MATLAB Integration (Deferred) . . . 25
8.3 Video Codec . . . 25
8.4 Potential Enhancements . . . 26

References **27**

Chapter 1

Introduction to Audio Codecs

1.1 What Is a Codec?

The word *codec* is a portmanteau of **co**der and **dec**oder. A codec is a system—implemented in software, hardware, or both—that *encodes* a data stream into a compressed or alternative representation and *decodes* it back into a form suitable for playback or further processing. In the audio domain a codec transforms raw pulse-code-modulated (PCM) samples into a more compact format and later reconstructs an approximation (or, in the lossless case, an exact copy) of the original signal.

1.2 Why Codecs Matter

Uncompressed audio occupies substantial storage and bandwidth. A single minute of CD-quality stereo audio (44.1 kHz, 16-bit) requires approximately 10.1 MB. Codecs reduce this footprint, enabling:

- **Efficient storage** – music libraries, archives, and embedded devices benefit from smaller files.
- **Reduced bandwidth** – streaming services and real-time communication rely on codecs to deliver audio within network constraints.
- **Perceptual quality control** – many codecs let the user trade file size against perceived audio quality.

1.3 Lossless vs. Lossy Compression

Audio codecs fall into two broad categories:

Lossless codecs (e.g. FLAC, ALAC) reconstruct the original samples exactly. Compression ratios are typically modest (40–60 % of original size).

Lossy codecs (e.g. MP3, AAC, Opus) discard information deemed inaudible by psychoacoustic models, achieving much higher compression at the cost of irreversible quality loss.

A third, less formal category is *perceptually lossless* compression: the reconstruction is not bit-exact, but the differences are designed to be imperceptible to human listeners. The KEE codec described in this document targets this category.

1.4 Key Concepts

1.4.1 Sampling Rate and the Nyquist Theorem

An analog audio signal is digitised by measuring its amplitude at regular intervals. The number of measurements per second is the *sampling rate* f_s (in Hz). The Nyquist–Shannon sampling theorem [1, 2] states that a signal can be perfectly reconstructed from its samples if and only if it contains no frequency component at or above $f_s/2$, the *Nyquist frequency*.

$$f_{\text{Nyquist}} = \frac{f_s}{2} \tag{1.1}$$

1.4.2 Quantisation and Bit Depth

Each sample is stored as an integer with a fixed number of bits (the *bit depth*). CD audio uses 16 bits per sample, giving $2^{16} = 65\,536$ discrete amplitude levels. Higher bit depths yield greater dynamic range at the expense of larger files.

1.4.3 The Fast Fourier Transform (FFT)

The FFT is an efficient algorithm for computing the Discrete Fourier Transform (DFT), which decomposes a time-domain signal into its constituent frequency components [3]. The KEE codec uses the FFT to determine the highest significant frequency in the input, enabling automatic selection of a reduced sampling rate.

1.5 Common Audio Codecs

Table 1.1 lists well-known audio codecs for context.

Table 1.1: Selected audio codecs and their characteristics.

Codec	**Type**	**Typical Use**	**Approach**
PCM (WAV)	Uncompressed	Editing, archival	None
FLAC	Lossless	Archival, audiophile	Predictive coding
MP3	Lossy	Music distribution	Psychoacoustic model
AAC	Lossy	Streaming, mobile	MDCT-based
Opus	Lossy	VoIP, streaming	Hybrid SILK/CELT
KEE	**Perceptually lossless**	**Custom project**	**FFT-based downsampling**

1.6 Where the KEE Codec Fits

The KEE codec takes a distinct approach: rather than applying psychoacoustic masking or transform coding, it analyses the frequency content of the input via the FFT, identifies the highest significant frequency, and *downsamples* the signal to a rate just sufficient to represent that content (by the Nyquist theorem). On the decoder side it *upsamples* and applies a reconstruction filter to restore the original sampling rate. The result is a file that is smaller than the original yet preserves all perceptually relevant frequency content.

Chapter 2

Core Components of an Audio Codec

This chapter describes the fundamental building blocks that a downsampling-based audio codec requires. Each block maps to one or more modules in the KEE implementation.

2.1 Signal Analysis (Frequency Domain)

To decide how much the sampling rate can be reduced, the encoder must determine which frequencies carry significant energy. The FFT converts the time-domain signal $x[n]$ of length N into frequency-domain coefficients $X[k]$:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j\,2\pi kn/N}, \qquad k = 0, 1, \ldots, N-1 \tag{2.1}$$

Only frequency bins whose magnitude $|X[k]|$ exceeds a configurable threshold are considered significant.

2.2 Filtering

Two filtering stages are involved:

1. **Anti-aliasing filter (encoder side).** Before decimation a low-pass filter removes frequency content above $f_{s,\text{new}}/2$ to prevent aliasing artefacts.
2. **Reconstruction filter (decoder side).** After upsampling a low-pass filter suppresses spectral images (copies of the baseband introduced by zero-insertion), retaining only the original signal band.

Both filters are implemented as Finite Impulse Response (FIR) filters designed with the window method (`scipy.signal.firwin`).

2.3 Downsampling and Upsampling

2.3.1 Downsampling (Decimation)

Decimation by an integer factor D means keeping every D-th sample:

$$y[n] = x[nD] \tag{2.2}$$

The effective sampling rate drops from f_s to f_s/D.

2.3.2 Upsampling (Interpolation)

Upsampling by D inserts $D-1$ zeros between each sample and then filters the result to reconstruct intermediate values. Polyphase resampling (`scipy.signal.resample_poly`) implements this efficiently without explicitly creating the zero-stuffed signal.

2.4 Metadata and Container Formats

A container format wraps the compressed audio payload together with descriptive metadata (original sampling rate, number of channels, duration, etc.) so that the decoder has all the information it needs. The KEE project defines a custom binary container with the `.kee` extension.

2.5 Integrity Verification

A SHA-256 checksum computed over the downsampled payload is stored in the metadata block. On decode the checksum can be recomputed and compared to detect corruption or tampering.

2.6 Generic Encode / Decode Pipeline

Figure 2.1 shows the generic data flow that the KEE codec follows.

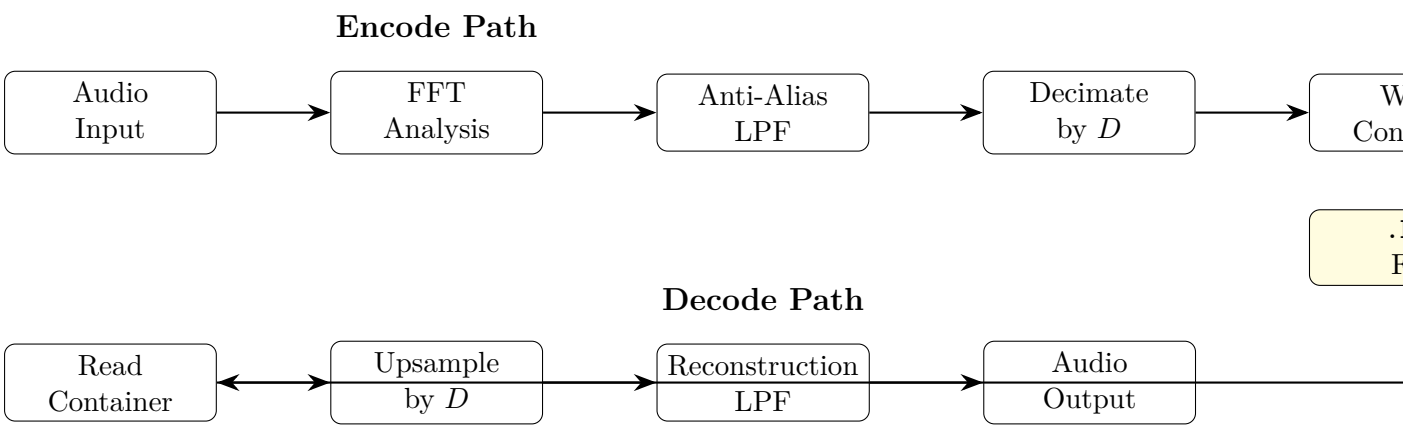

Figure 2.1: Generic encode and decode pipeline of the KEE codec.

Chapter 3

KEE CODEC Project Overview

3.1 Project Goals and Motivation

The KEE Audio CODEC project set out to build a fully functional, *perceptually lossless* audio codec from scratch. The primary goals were:

1. Demonstrate the practical application of the Nyquist theorem and FFT-based frequency analysis for audio compression.
2. Design a custom binary container format (`.kee`) with embedded metadata and integrity verification.
3. Provide both a command-line interface and a web-based graphical interface for encoding and decoding.
4. Maintain a clean, modular Python codebase with comprehensive tests.

3.2 High-Level Architecture

Figure 3.1 depicts the high-level architecture of the project, showing the three main interfaces (library, CLI, web) sharing the core codec modules.

3.3 Technology Stack

Table 3.1: Technologies used in the KEE codec project.

Category	**Technology**	**Role**
Language	Python 3	Primary implementation language
Numerics	NumPy [4]	Array operations, FFT
DSP	SciPy [5]	FIR filter design, polyphase resampling
Audio I/O	soundfile [6]	Read/write WAV, FLAC, OGG, AIFF
Web	Flask [7]	Backend REST API and static file serving
Testing	pytest	Unit and integration test runner

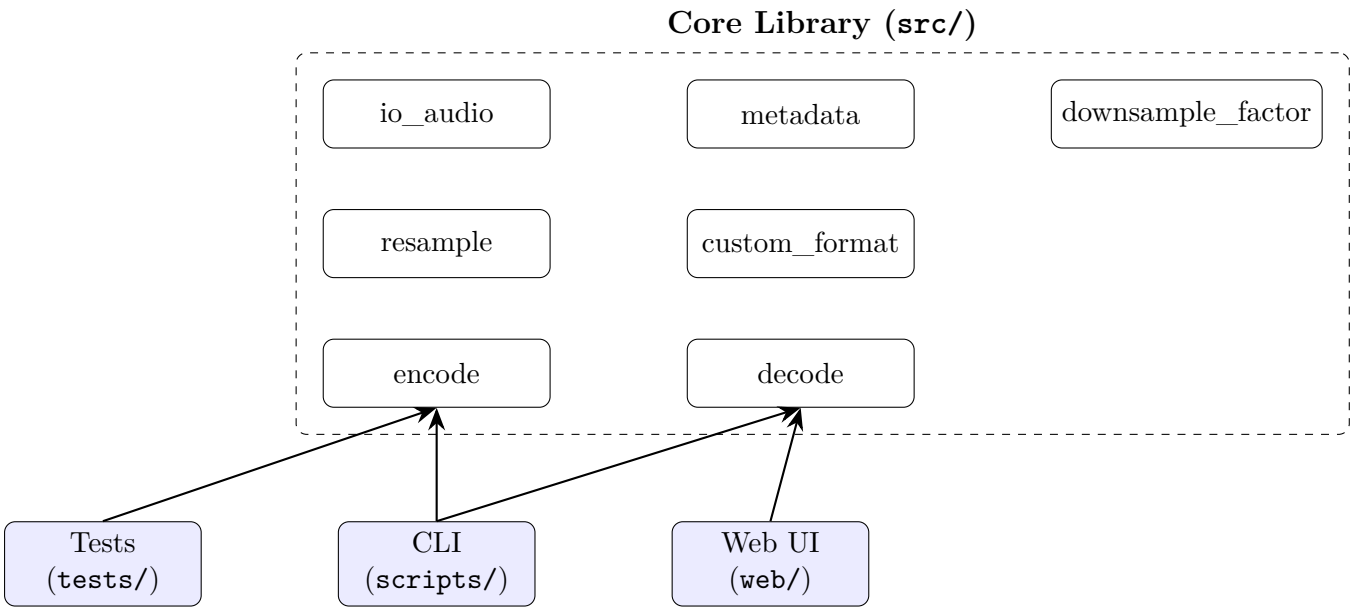

Figure 3.1: High-level project architecture.

3.4 Project Directory Structure

```
codec-project/
  src/
    io_audio.py            - Audio file I/O
    metadata.py            - Metadata schema + checksum
    downsample_factor.py   - FFT-based auto D computation
    resample.py            - LPF, decimate, upsample
    custom_format.py       - .kee reader / writer
    encode.py              - Encoding pipeline
    decode.py              - Decoding pipeline
  scripts/
    cli_encode.py          - CLI encoder
    cli_decode.py          - CLI decoder
  web/
    app.py                 - Flask backend
    templates/index.html   - Web UI
    static/style.css       - Styles
    static/app.js          - Frontend logic
  tests/
    test_format.py         - .kee format + metadata tests
    test_codec.py          - Encode/decode round-trip tests
  requirements.txt
  README.md
```

Listing 3.1: Project directory layout.

Chapter 4

Detailed Component Descriptions

This chapter examines each module in the `src/` package.

4.1 Audio I/O (`src/io_audio.py`)

This module provides two functions and one data class for loading and saving audio files.

AudioData dataclass. Holds all relevant properties of a loaded audio file:

```
@dataclass
class AudioData:
    samples: np.ndarray    # (n_samples,) mono or (n_samples, ch
        ) multi
    sample_rate: int
    channels: int
    subtype: str           # e.g. "PCM_16"
    n_samples: int
    duration: float
```

Listing 4.1: AudioData dataclass.

load_audio. Reads any format supported by `libsndfile` (WAV, FLAC, OGG, AIFF) via the `soundfile` library, returning an `AudioData` instance with 64-bit floating-point samples.

save_audio. Writes a NumPy array to a WAV file at the specified sample rate using 16-bit PCM encoding.

4.2 Metadata (`src/metadata.py`)

CodecMetadata dataclass. Captures all information the decoder needs to restore the original signal:

```
@dataclass
class CodecMetadata:
    fs_original: int
    fs_new: int
```

```
    downsample_factor: int
    channels: int
    duration: float
    n_samples_original: int
    checksum: str = ""
```

Listing 4.2: CodecMetadata dataclass.

Table 4.1: Metadata field descriptions.

Field	Description
`fs_original`	Original sampling rate of the input audio (Hz).
`fs_new`	Reduced sampling rate after downsampling (Hz).
`downsample_factor`	Integer decimation factor D.
`channels`	Number of audio channels (1 = mono, 2 = stereo).
`duration`	Duration of the original audio in seconds.
`n_samples_original`	Total number of samples in the original signal.
`checksum`	SHA-256 hex digest of the downsampled payload.

The metadata is serialised to compact JSON via `to_bytes()` and deserialised via the class method `from_bytes()`.

compute_checksum. Computes a SHA-256 digest over the raw byte representation of the sample array, providing an integrity check that the decoder can verify.

4.3 Auto Downsample Factor (`src/downsample_factor.py`)

When the user does not supply a downsampling factor, the codec computes one automatically using the following FFT-based algorithm:

1. Compute the real FFT of the (mono) input signal.
2. Identify all frequency bins whose magnitude exceeds a configurable amplitude threshold τ (default $\tau = 500$).
3. Let `max_bin` be the index of the highest significant bin.
4. Compute the corresponding physical frequency:
$$f_{\max} = \texttt{max_bin} \times \frac{f_s}{N} \tag{4.1}$$
where f_s is the original sampling rate and N is the number of samples.
5. By the Nyquist theorem, the minimum sampling rate that preserves all content up to $f_{\max}$ is:
$$f_{s,\text{new}} = 2\, f_{\max} \tag{4.2}$$
6. The integer downsampling factor is:
$$D = \left\lceil \frac{f_s}{f_{s,\text{new}}} \right\rceil \tag{4.3}$$

7. $f_{s,\text{new}}$ is recalculated as $\lceil f_s/D \rceil$ to ensure consistency with the chosen integer D.

```
fft_vals = np.fft.rfft(mono)
magnitudes = np.abs(fft_vals)
significant_bins = np.where(magnitudes >= threshold)[0]
max_bin = int(significant_bins[-1])
f_max = max_bin * (fs_original / n)
fs_new = int(math.ceil(2 * f_max))
d = math.ceil(fs_original / fs_new)
```

Listing 4.3: Core of the auto downsample factor computation.

4.4 Resampling (`src/resample.py`)

This module provides three functions that implement the signal-processing core of both the encoder and the decoder.

4.4.1 Low-Pass Filter

```
def lowpass_filter(samples, cutoff_hz, fs, numtaps=101):
    nyquist = fs / 2.0
    normalized_cutoff = cutoff_hz / nyquist
    coeffs = firwin(numtaps, normalized_cutoff)
    return lfilter(coeffs, 1.0, samples)
```

Listing 4.4: FIR low-pass filter.

A 101-tap FIR filter is designed using the window method (`scipy.signal.firwin`). Multi-channel audio is handled by filtering each channel independently.

4.4.2 Decimate

Decimation takes every D-th sample from the (already filtered) signal:

```
def decimate(samples, factor):
    return samples[::factor]
```

Listing 4.5: Decimation function.

4.4.3 Upsample

Upsampling by factor D is performed via polyphase resampling (`scipy.signal.resample_poly`), which is both memory-efficient and numerically stable:

```
def upsample(samples, factor):
    return resample_poly(samples, factor, 1)
```

Listing 4.6: Polyphase upsampling.

4.5 Custom Format (`src/custom_format.py`)

The `.kee` file format is a simple binary container. Its layout is detailed in Chapter 6; the key implementation points are:

write_kee. Serialises the metadata to JSON bytes, quantises the floating-point samples to 16-bit signed integers (little-endian), and writes the magic bytes, version, metadata length, metadata, and payload sequentially.

```
def write_kee(path, metadata, samples):
    meta_bytes = metadata.to_bytes()
    pcm_int16 = (np.clip(samples, -1.0, 1.0) * 32767).astype("<
        i2")
    with open(str(path), "wb") as f:
        f.write(MAGIC)                              # b"KEE\x00"
        f.write(struct.pack("<H", FORMAT_VERSION)) # uint16 LE
        f.write(struct.pack("<I", len(meta_bytes)))# uint32 LE
        f.write(meta_bytes)
        f.write(pcm_int16.tobytes())
```

Listing 4.7: Writing a `.kee` file.

read_kee. Reads and validates the magic bytes and version, deserialises the metadata JSON, and converts the PCM payload back to 64-bit floating-point samples. Multi-channel audio is reshaped according to the channel count in the metadata.

4.6 Encoding Pipeline (`src/encode.py`)

The `encode()` function orchestrates the full encoding workflow:

1. **Load** the input audio via `load_audio()`.
2. **Determine** D: use the caller-supplied factor, or compute it automatically via `compute_auto_downsample_factor()`.
3. **Filter**: apply a low-pass filter with cutoff $f_{s,\text{new}}/2$.
4. **Decimate**: take every D-th sample.
5. **Checksum**: compute SHA-256 over the downsampled payload.
6. **Build metadata**: populate a `CodecMetadata` instance.
7. **Write**: serialise everything to a `.kee` file via `write_kee()`.

```
def encode(audio_path, output_path, downsample_factor=None,
           threshold=500.0):
    audio = load_audio(audio_path)
    if downsample_factor is not None:
        d = max(1, downsample_factor)
        fs_new = int(math.ceil(audio.sample_rate / d))
```

```
    else:
        d, fs_new = compute_auto_downsample_factor(
            audio.samples, audio.sample_rate, threshold)
    if d > 1:
        filtered = lowpass_filter(audio.samples,
                                  fs_new / 2.0,
                                  audio.sample_rate)
        downsampled = decimate(filtered, d)
    else:
        downsampled = audio.samples
    checksum = compute_checksum(downsampled)
    metadata = CodecMetadata(
        fs_original=audio.sample_rate, fs_new=fs_new,
        downsample_factor=d, channels=audio.channels,
        duration=audio.duration,
        n_samples_original=audio.n_samples,
        checksum=checksum)
    write_kee(output_path, metadata, downsampled)
    return metadata
```

Listing 4.8: Encoding pipeline (simplified).

4.7 Decoding Pipeline (`src/decode.py`)

The `decode()` function reverses the encoding:

1. **Read** the `.kee` file via `read_kee()`.
2. **Upsample** by D using polyphase resampling.
3. **Reconstruct**: apply a low-pass filter with cutoff $f_{s,\text{new}}/2$ to remove spectral images.
4. **Trim** the restored signal to the original sample count (upsampling may introduce a few extra samples).
5. **Save** the result as a WAV file at the original sampling rate.

```
def decode(kee_path, output_audio_path):
    metadata, samples = read_kee(kee_path)
    d = metadata.downsample_factor
    if d > 1:
        upsampled = upsample(samples, d)
        restored = lowpass_filter(upsampled,
                                  metadata.fs_new / 2.0,
                                  metadata.fs_original)
    else:
        restored = samples
    n_target = metadata.n_samples_original
    restored = restored[:n_target]
    save_audio(output_audio_path, restored,
               metadata.fs_original)
```

```
    return metadata
```

Listing 4.9: Decoding pipeline (simplified).

Chapter 5

Development Phases

The project was developed in five phases, progressing from the core algorithm to user-facing interfaces and finally to quality assurance and documentation.

5.1 Phase 1: Core CODEC

Phase 1 established the foundational encode/decode logic.

5.1.1 Stage 1.1 – Project Setup and Dependencies

The project repository was initialised with a standard Python layout: `src/`, `scripts/`, `tests/`, and `web/` directories. A `requirements.txt` was created listing the core dependencies: `numpy`, `scipy`, `soundfile`, and `flask`. A virtual environment (`venv`) was recommended for isolation.

5.1.2 Stage 1.2 – Input Handling and Analysis

Stage 1.2a – File I/O. The `io_audio` module (Section 4.1) was implemented, providing `load_audio()` and `save_audio()` with support for WAV, FLAC, OGG, and AIFF via the `soundfile` library.

Stage 1.2b – Metadata Extraction. The `CodecMetadata` dataclass (Section 4.2) was defined to capture all properties needed by the decoder: original and new sampling rates, downsampling factor, channel count, duration, original sample count, and a SHA-256 checksum. JSON serialisation and deserialisation methods were added.

5.1.3 Stage 1.3 – Downsampling (Encoding)

Stage 1.3a – Auto Downsample Factor. The FFT-based algorithm described in Section 4.3 was implemented in `downsample_factor.py`. When the user omits a downsample factor, the encoder computes D automatically using the equations in (4.1)–(4.3).

Stage 1.3b – Low-Pass Filter and Decimate. The anti-aliasing FIR filter and decimation functions were implemented in `resample.py` (Section 4.4).

Stage 1.3c – User Override. The encoding pipeline accepts an optional `downsample_factor` parameter; when provided, it bypasses the FFT analysis and uses the caller-supplied value.

Stage 1.3d – Metadata Storage. All computed parameters (f_s, $f_{s,\text{new}}$, D, channels, duration) are written into the `.kee` metadata block so the decoder is self-contained.

5.1.4 Stage 1.4 – Custom Format Design

Stage 1.4a – Layout. The binary layout of the `.kee` format was designed (see Chapter 6): magic bytes, format version, metadata length, JSON metadata, and raw PCM payload.

Stage 1.4b – Serialisation. `write_kee()` and `read_kee()` were implemented in `custom_format.py` (Section 4.5).

5.1.5 Stage 1.5 – Encoding Pipeline

The `encode()` function (Section 4.6) was built to chain all encoder stages into a single call: load → analyse → filter → decimate → write `.kee`.

5.1.6 Stage 1.6 – Upsampling (Decoding)

Polyphase upsampling and a reconstruction low-pass filter were implemented (Section 4.4). The cutoff frequency matches the one used during encoding ($f_{s,\text{new}}/2$), ensuring that only the baseband is retained.

5.1.7 Stage 1.7 – Decoding Pipeline

The `decode()` function (Section 4.7) chains the decoder stages: read `.kee` → upsample → filter → trim → save audio.

5.2 Phase 2: CLI and Scripts

5.2.1 Stage 2.1 – Command-Line Interface

Two CLI scripts were created using Python's `argparse`:

- `scripts/cli_encode.py` – accepts an input audio file, an optional output path, an optional downsample factor (`-d`), and an optional FFT threshold (`--threshold`). Prints encoding statistics (original size, compressed size, ratio, D used).
- `scripts/cli_decode.py` – accepts a `.kee` file and an optional output path. Restores the audio and prints metadata.

Example usage:

```python
# Auto downsample factor
python -m scripts.cli_encode input.wav -o output.kee

# User-supplied D = 4
python -m scripts.cli_encode input.wav -o output.kee -d 4

# Decode
python -m scripts.cli_decode output.kee -o restored.wav
```

Listing 5.1: CLI encoding and decoding examples.

5.2.2 Stage 2.2 – Metafile-Only Option

The `.kee` file itself serves as the "metafile": it contains both the metadata and the compressed payload. The CLI naturally supports producing only the `.kee` (encode) and later consuming it (decode) as separate operations.

5.3 Phase 3: Web UI

5.3.1 Stage 3.1 – Landing and Encode/Decode Entry

A single-page web interface was built with two main panels—**Encode** and **Decode**—accessible from a landing page with a title and brief description.

5.3.2 Stage 3.2 – Encode Flow

The encode panel provides:

1. A file browser to select an audio file (WAV, FLAC, OGG, AIFF; max 50 MB).
2. An optional downsample factor input ("Auto" by default).
3. An **Upload & Encode** button that sends the file to the server.
4. Upon success, a **Download .kee** button appears.

5.3.3 Stage 3.3 – Decode Flow

The decode panel offers two input methods:

1. **Browse** – upload a `.kee` file from disk.
2. **Use last encode** – a checkbox that reuses the `.kee` file produced in the current session (kept as a blob in the browser).

After decoding, a **Download restored WAV** button appears.

5.3.4 Stage 3.4 – Backend

The Flask backend (`web/app.py`) exposes two endpoints:

- `POST /encode` – receives a multipart audio file and an optional downsample factor; runs the encode pipeline and returns the `.kee` file.
- `POST /decode` – receives a `.kee` file; runs the decode pipeline and returns the restored WAV.

The `.kee` blob is stored client-side to avoid server-side session management, simplifying deployment.

5.4 Phase 4: Quality and Robustness

5.4.1 Stage 4.1 – Validation and Tests

Two test modules were created:

- `tests/test_format.py` – unit tests for `.kee` format read/write round-trips and metadata serialisation.
- `tests/test_codec.py` – integration tests that encode a synthetic signal, decode it, and verify that duration and basic statistics are preserved.

Tests are run with `pytest`:

```
pytest tests/ -v
```

5.4.2 Stage 4.2 – Error Handling and Limits

- Unsupported file formats and oversized uploads are rejected with clear error messages in both the CLI and the web API.
- The `read_kee()` function validates the magic bytes and format version, raising `ValueError` for corrupted or unrecognised files.
- The web UI enforces a 50 MB upload limit.

5.5 Phase 5: Documentation and Enhancements

5.5.1 Stage 5.1 – Documentation

A comprehensive `README.md` was written covering:

- Project goal and quick-start instructions.
- CLI usage with flag descriptions.
- Web UI launch instructions and workflow.

- `.kee` format specification.
- Algorithmic overview of encoding and decoding.
- Project structure summary.

5.5.2 Stage 5.2 – Backlog Alignment and Future Work

The development plan identified several items for future consideration:

- **MATLAB integration** – optional MATLAB scripts matching the `.kee` format, callable via `matlab.engine` or run independently. This was deferred as non-essential to the core project.
- **Video codec** – noted as a separate future project that would reuse metadata and container concepts.

Chapter 6

The `.kee` File Format Specification

6.1 Binary Layout

Table 6.1 shows the byte-level layout of a `.kee` file.

Table 6.1: Binary layout of the `.kee` file format.

Offset	Size	Field	Description
0	4 B	Magic bytes	`KEE\x00` – identifies the format.
4	2 B	Version	Format version as `uint16` (LE). Currently 1.
6	4 B	Meta length	Length M of the metadata block (`uint32` LE).
10	M B	Metadata	JSON-encoded `CodecMetadata` (UTF-8).
$10 + M$	variable	Payload	Raw PCM samples, signed 16-bit integers (LE).

6.2 Metadata JSON Schema

The metadata block is a compact JSON object (no whitespace) with the fields listed in Table 4.1 (page 11). An example:

```
{"fs_original":44100,"fs_new":11025,"downsample_factor":4,
 "channels":1,"duration":3.5,"n_samples_original":154350,
 "checksum":"a3f1...b7e2"}
```

Listing 6.1: Example metadata JSON.

6.3 Payload Format

Audio samples are quantised to signed 16-bit integers (range $[-32\,767, +32\,767]$) in little-endian byte order. Floating-point samples are clipped to $[-1, 1]$ before quantisation:

$$s_{\text{int16}} = \text{clip}(s_{\text{float}},\ -1,\ 1) \times 32\,767 \tag{6.1}$$

For multi-channel audio the samples are interleaved (channel 0 sample 0, channel 1 sample 0, channel 0 sample 1, ...).

6.4 Format Diagram

Figure 6.1 illustrates the layout visually.

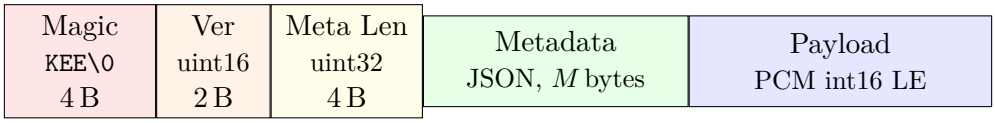

Figure 6.1: Visual layout of the `.kee` binary format.

Chapter 7

Usage Guide

7.1 Installation

```
# Clone or copy the project
cd codec-project

# Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate           # Windows
# source venv/bin/activate      # macOS / Linux

# Install dependencies
pip install -r requirements.txt
```

Listing 7.1: Setting up the project.

7.2 CLI Encoding

```
# Auto downsample factor (FFT-based)
python -m scripts.cli_encode input.wav -o output.kee

# User-supplied downsample factor
python -m scripts.cli_encode input.wav -o output.kee -d 4

# Custom amplitude threshold for auto D
python -m scripts.cli_encode input.wav -o output.kee --
    threshold 300
```

Listing 7.2: Encoding with the CLI.

7.3 CLI Decoding

```
python -m scripts.cli_decode output.kee -o restored.wav
```

Listing 7.3: Decoding with the CLI.

Table 7.1: CLI encoder flags.

Flag	Description
`-o`	Output `.kee` path (default: `<input_stem>.kee`).
`-d`	Downsample factor (omit for FFT-based auto).
`--threshold`	Amplitude threshold for auto-D (default: 500).

Table 7.2: CLI decoder flags.

Flag	Description
`-o`	Output audio path (default: `<input_stem>_restored.wav`).

7.4 Web UI

```
python web/app.py
# Open http://localhost:5000 in a browser
```

Listing 7.4: Starting the web server.

7.4.1 Encode Workflow

1. Select the **Encode** panel.
2. Browse and select an audio file (WAV, FLAC, OGG, or AIFF; max 50 MB).
3. Optionally enter a downsample factor or leave it as "Auto".
4. Click **Upload & Encode**.
5. When encoding completes, click **Download .kee** to save the compressed file.

7.4.2 Decode Workflow

1. Select the **Decode** panel.
2. Either browse and upload a `.kee` file, or check *"Use the metafile from the last encode"* to reuse the one just produced.
3. Click **Decode**.
4. When decoding completes, click **Download restored WAV**.

7.5 Running Tests

```
pip install pytest
pytest tests/ -v
```

Listing 7.5: Running the test suite.

Chapter 8

Conclusion and Future Work

8.1 Summary

The KEE Audio CODEC project successfully demonstrates a complete audio compression pipeline built on fundamental digital signal processing principles:

- FFT-based frequency analysis enables automatic, content-aware selection of the downsampling factor.
- FIR filtering ensures alias-free decimation and artefact-free reconstruction.
- A custom binary container (`.kee`) encapsulates metadata and payload with integrity verification via SHA-256.
- A modular Python codebase supports three access methods: programmatic API, command-line tools, and a web interface.
- Comprehensive tests validate both the format handling and the end-to-end encode/decode round-trip.

8.2 MATLAB Integration (Deferred)

The development plan included optional MATLAB scripts that mirror the Python encode/decode logic and read/write the same `.kee` format. This was deferred as non-essential; the Python implementation is fully self-contained. Future work could add MATLAB Live Scripts callable via `matlab.engine` from Python, or usable independently for comparison and educational purposes.

8.3 Video Codec

The development plan notes a video codec as a separate future project. It would reuse concepts from the audio codec—metadata schemas, custom container design—while introducing frame-based processing and temporal compression techniques.

8.4 Potential Enhancements

Several enhancements could improve the codec further:

- **Adaptive thresholding** – replace the fixed amplitude threshold with a percentile-based or SNR-based criterion for determining significant frequency bins.
- **Perceptual quality metrics** – integrate objective quality measures (e.g. PESQ, POLQA, or ViSQOL) into the test suite for automated quality assessment.
- **Multi-band compression** – split the spectrum into sub-bands and apply different downsampling factors to each, potentially achieving higher compression on signals with sparse high-frequency content.
- **Streaming support** – extend the `.kee` format to support chunked encoding for real-time streaming applications.
- **Lossless mode** – add an option to store the original samples without downsampling (i.e. $D = 1$) but still benefit from the metadata and container structure.

Bibliography

[1] H. Nyquist, "Certain Topics in Telegraph Transmission Theory," *Transactions of the American Institute of Electrical Engineers*, vol. 47, no. 2, pp. 617–644, 1928.

[2] C. E. Shannon, "Communication in the Presence of Noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949.

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.

[4] NumPy Developers, "NumPy – The fundamental package for scientific computing with Python." https://numpy.org, 2024.

[5] SciPy Developers, "SciPy – Fundamental algorithms for scientific computing in Python." https://scipy.org, 2024.

[6] B. Bechtold, "python-soundfile – SoundFile is an audio library based on libsndfile." https://python-soundfile.readthedocs.io, 2024.

[7] Pallets Projects, "Flask – A lightweight WSGI web application framework." https://flask.palletsprojects.com, 2024.